DESARROLLO DE INTERFACES · 2º DAM

El patrón Modelo-Vista-Controlador en JavaFX

Qué es el MVC, ejemplos resueltos y tareas para clase

Punto de partida: una calculadora sencilla

El usuario escribe dos números, pulsa + y ve el resultado.



Su código tiene que hacer tres cosas

1. **Dibujar la ventana:** campos, botón y etiqueta
2. **Reaccionar** cuando se pulsa el botón
3. **Hacer la cuenta:** la lógica de la suma

Siguiente diapositiva: cómo queda el código si metemos las tres cosas en una sola clase.

El problema: una clase que lo hace todo

```
public class Calculadora extends Application {
    public void start(Stage ventana) {
        TextField a = new TextField();    // 1. ventana
        TextField b = new TextField();    // 1. ventana
        Label res = new Label();          // 1. ventana
        Button sumar = new Button("+");   // 1. ventana

        sumar.setOnAction(e → {           // 2. reacción
            int x = Integer.parseInt(a.getText());
            int y = Integer.parseInt(b.getText());
            res.setText("" + (x + y));    // 3. lógica
        });
    }
}
```

Las tres tareas están mezcladas en la misma clase. Consecuencias:

- No se puede probar la lógica sin abrir la ventana
- No se puede cambiar la ventana sin tocar la lógica
- Los datos viven dentro de los campos de texto
- Cualquier cambio toca la misma clase

¿Qué es el Modelo-Vista-Controlador?

Un **patrón de arquitectura**: separa la aplicación en tres partes, cada una con una sola responsabilidad.

Modelo

Los datos y las reglas.

¿Qué información hay y qué reglas se cumplen?

Vista

Lo que ve y toca el usuario.

¿Cómo se muestran los datos?

Controlador

El intermediario entre ambos.

¿Qué pasa cuando el usuario actúa?

Qué hace y qué no hace cada parte

Parte	Qué hace	Qué NO hace	En JavaFX
Modelo	Guarda los datos, aplica las reglas y avisa de los cambios	No conoce la vista. No importa javafx.scene	Clases normales, propiedades y ObservableList
Vista	Pinta los datos y avisa de lo que hace el usuario	No calcula ni decide	BorderPane, Label, Button, TableView… o un FXML
Controlador	Recibe las acciones, llama al modelo y actualiza la vista	No guarda datos ni dibuja	Clase con setOnAction o métodos @FXML

La columna **«Qué NO hace»** es la más importante: es lo que mantiene el código separado.

El flujo de una acción

1. El usuario pulsa un botón de la **vista**
2. El **controlador** recibe el evento
3. El **modelo** cambia sus datos
4. La **vista** muestra el nuevo estado

Quién conoce a quién: el controlador conoce al modelo y a la vista. El modelo no conoce a nadie.

La vista aparece al principio y al final, pero nunca habla directamente con el modelo.

Cómo se entera la vista: dos variantes

A · El controlador refresca

Después de cambiar el modelo, el controlador le pide el valor y llama a vista.mostrar(...).

```
modelo.incrementar();
vista.mostrar(modelo.getValor());
```

Más simple. La usa el contador.

B · El modelo avisa (Observer)

El modelo usa propiedades y listas observables. La vista se enlaza con bind() y se actualiza sola.

```
lblTotal.textProperty().bind(
    total.asString("Total: %.2f €"));
```

Varias vistas sincronizadas. La usa la lista de la compra.

Ventajas e inconvenientes

Ventajas

- Cambiar la interfaz sin tocar la lógica
- Probar el modelo con JUnit, sin ventanas
- Reutilizar el modelo, con o sin FXML
- Trabajar en equipo por capas
- Saber dónde buscar cada fallo

Inconvenientes

- Más clases, incluso en apps pequeñas
- Cuesta un poco al principio
- Riesgo de controladores enormes

Analogía: la cocina es el modelo, la sala y la carta son la vista, y el camarero es el controlador.

MVC en tecnologías reales

Tecnología	Modelo	Vista	Controlador
JavaFX (este curso)	Clases con propiedades y ObservableList	Panel en código o fichero FXML	Clase con setOnAction o @FXML
Swing	Clases Java, TableModel	JFrame, JTable	ActionListener
Spring MVC (web)	Entidades y servicios	Plantillas HTML	Clase @Controller
Android (MVVM)	Datos y repositorios	Layout XML	ViewModel

La idea es la misma en todas: cambian los nombres de las piezas, no el reparto de responsabilidades.

Una aplicación JavaFX por dentro



> Ejecuta siempre **Main**. Si ejecutas App desde el IDE: «JavaFX runtime components are missing».

```
public class Main {
    public static void main(String[] args) {
        Application.launch(App.class, args);
    }
}

public class App extends Application {
    @Override
    public void start(Stage ventana) {
        // crear modelo, vista y controlador
        ventana.setScene(new Scene(vista));
        ventana.show();
    }
}
```

PARTE 2

Ejemplos resueltos

Cinco ejemplos del proyecto mvc-javafx, de menos a más

1 · Contador

1b · Contador con FXML

2 · Login

3 · Lista de la compra

4 · TableView

Contador: modelo y vista

MODELO · Contador.java

```
public class Contador {
    private int valor;

    public void incrementar() { valor++; }
    public void reiniciar() { valor = 0; }
    public int getValor() { return valor; }
}
```

VISTA · VistaContador.java

```
public class VistaContador extends BorderPane {
    private final Label lblValor = new Label("0");
    private final Button btnIncrementar = new Button("+");
    private final Button btnReset = new Button("Reset");
    public void mostrar(int valor) {
        lblValor.setText(String.valueOf(valor));
    }
    public void alPulsarIncrementar(Runnable accion) {
        btnIncrementar.setOnAction(e → accion.run());
    }
}
```

El modelo no importa nada de JavaFX. La vista no tiene ni un if ni un cálculo.

Contador: controlador y arranque

```
public class ControladorContador {
    public ControladorContador(Contador modelo,
                               VistaContador vista) {
        this.modelo = modelo; this.vista = vista;
        vista.alPulsarIncrementar(() → incrementar());
        vista.alPulsarReset(() → reiniciar());
        refrescarVista();      // estado inicial
    }
    private void incrementar() {
        modelo.incrementar();  // 1) cambiar el modelo
        refrescarVista();      // 2) actualizar la vista
    }
    private void refrescarVista() {
        vista.mostrar(modelo.getValor());
    }
}
```

APP · start()

```
Contador modelo = new Contador();
VistaContador vista = new VistaContador();
new ControladorContador(modelo, vista);
ventana.setScene(new Scene(vista));
```

Variante A: el controlador cambia el modelo y luego refresca la vista.

App solo crea las tres piezas y las conecta: no tiene lógica.

El mismo contador con FXML

VISTA · VistaContador.fxml

```
<BorderPane fx:controller="…ControladorContador">
  <center>
    <Label fx:id="lblValor" text="0"/>
  </center>
  <bottom>
    <HBox alignment="CENTER" spacing="10">
      <Button text="+" onAction="#incrementar"/>
      <Button text="Reset" onAction="#reiniciar"/>
    </HBox>
  </bottom>
</BorderPane>
```

CONTROLADOR · @FXML

```
public class ControladorContador {
    @FXML private Label lblValor;

    @FXML private void initialize() {
        refrescarVista();
    }
    @FXML private void incrementar() {
        modelo.incrementar();
        refrescarVista();
    }
}
```

fx:id → atributo @FXML

onAction="#x" → método @FXML

Reutiliza el modelo Contador

Pantalla de login

Modelo

UsuarioService guarda los usuarios y decide con validar() si el login es correcto.

Vista

TextField, PasswordField, botón Entrar y mostrarMensaje(String).

Controlador

Lee los campos, pregunta al modelo y elige qué mensaje mostrar.

```
// ControladorLogin: la regla de negocio la decide el MODELO
if (servicio.validar(usuario, clave)) {
    vista.mostrarMensaje("¡Bienvenido, " + usuario + "!");
} else {
    vista.mostrarMensaje("Usuario o contraseña incorrectos");
    vista.limpiarClave();
}
```

Lista de la compra con dos vistas

Modelo

ListaCompra: una ObservableList de productos y el total como propiedad.

Vista 1

Un ListView con los productos.

Vista 2

Un Label con el total, enlazado con bind().

```
// Modelo: la lista avisa sola de cada cambio
productos.add(producto);

// App: cada vista se conecta a una parte del modelo
vistaProductos.mostrar(modelo.getProductos());  // lista.setItems(...)
vistaTotal.mostrar(modelo.totalProperty());     // textProperty().bind(...)
```

El controlador solo cambia el modelo: no llama a ningún refrescar().

Tablas que se actualizan solas: TableView

```
// Cada columna dice qué dato del Alumno enseña
TableColumn<Alumno, String> colNombre = new TableColumn<>("Nombre");
colNombre.setCellValueFactory(fila →
        new ReadOnlyObjectWrapper<>(fila.getValue().getNombre()));
tabla.getColumns().add(colNombre);

// La tabla OBSERVA la lista del modelo
tabla.setItems(grupo.getAlumnos());
```

Nombre	Edad	Nota
Ana García	19	8.5
Luis Pérez	21	6.0
Marta Ruiz	20	9.25
Pablo Sanz	22	7.0

La **TableView** es la vista y observa la **ObservableList** del modelo: si se añade un alumno, la fila aparece sola. Es el patrón Observer.

La nota media es otra propiedad del modelo, enlazada a un Label con bind(). Nadie llama a refrescar().

PARTE 3

Tareas para clase

De guiada a mini proyecto

1 · Conversor · 45 min

2 · Refactorizar · 1 h

3 · Cuenta bancaria · 1–2 h

4 · Agenda · 2–3 h

PARTE 3 · TAREA 1 · GUIADA · 45 MIN

Conversor de temperatura

Pasos

1. Crea los paquetes modelo, vista y controlador.
2. Modelo ConversorTemperatura con celsiusAFahrenheit y fahrenheitACelsius.
3. Prueba el modelo en un main: 100 °C dan 212 °F.
4. Vista con un campo, un botón Convertir y una etiqueta.
5. Controlador: si el texto no es un número, muestra un error.

Entregable y criterios

- Proyecto Maven con Main y App
- El modelo no importa javafx
- La vista no hace ningún cálculo
- Error con un texto que no es número
- Error por debajo del cero absoluto

PARTE 3 · TAREA 2 · REFACTORIZAR · 1 H

Separa la calculadora espagueti

Pasos

1. Abre la clase CalculadoraEspagueti, que lo hace todo.
2. Marca con tres colores qué es modelo, vista y controlador.
3. Extrae el modelo con sumar, restar, multiplicar y dividir.
4. Dividir entre cero lanza una excepción propia.
5. Extrae la vista y crea el controlador.

Entregable y criterios

- Funciona igual que el original
- Modelo sin javafx.scene ni stage
- Vista sin ningún if de operaciones
- Al menos un test JUnit del modelo

Pista: prueba «2 + 3 + 4 =». Encuentra el fallo con un test y arréglalo.

Cuenta bancaria con dos vistas

Pasos

1. CuentaBancaria con ingresar y retirar; si no hay saldo, excepción.
2. Saldo como propiedad y movimientos como ObservableList.
3. Vista de operaciones: importe, Ingresar y Retirar.
4. Vista de resumen: saldo, en rojo si baja de 100 €.
5. Tres tests JUnit del modelo.

Entregable y criterios

- Las vistas se actualizan solas
- El controlador no refresca nada
- Qué es «saldo bajo» lo decide el modelo
- El color rojo lo pone la vista
- Los tests no abren ninguna ventana

PARTE 3 · TAREA 4 · MINI PROYECTO · 2–3 H

Agenda de contactos con TableView

Pasos

1. Clase Contacto y modelo Agenda con una ObservableList.
2. Vista con TableView y un formulario.
3. Controlador con alta, baja y modificación.
4. Búsqueda por nombre.
5. Ampliación: guardar en fichero o hacer la vista en FXML.

Entregable y criterios

- No hay dos contactos con el mismo nombre
- La baja pide confirmación
- La búsqueda se mantiene tras cada cambio
- Código limpio